claude-windows / c1fcb4aa-537d-41fd-858e-53f93349bf5a

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\subagents\workflows\wf_41c8dc8a-8a4\agent-acd1464ed345cd57c.jsonl`
- SHA-256: `ac1e552916887b930c4efdaa45ce0e4aafdeca0c2f23f3bd22a26b5b766c3d09`
- Source modified: `2026-07-02T02:59:43+00:00`
- Imported at: `2026-07-05T16:48:24+00:00`
- project: `wf_41c8dc8a-8a4`
- session_id: `c1fcb4aa-537d-41fd-858e-53f93349bf5a`

Transcript

1. user (2026-07-02T02:58:05.141Z)

```
You are an adversarial verifier in a tech-debt audit. Your default stance: REFUTE. Today is
2026-07-02.
A finder claims the following about the repo at C:/Users/avidu/Projects/khelsutra-guru/khelsutra
(avidullu/khelsutra):

{
  "title": "Issue #111 still fully valid: refresh loses the in-flight job; progress static for
minutes",
  "category": "critical-fix",
  "severity": "P1",
  "file": "web/src/hooks/useIngestJob.ts",
  "line": 117,
  "evidence": "useIngestJob.ts:117-128 ? job_id lives only in the poll closure (no
localStorage/URL persistence); App.tsx:71-73 seeds state only from ?video= (set on done,
App.tsx:153). A refresh mid-job lands the user back on the ingest form while the server job
keeps running. Progress is the coarse free-text job.progress (useIngestJob.ts:106); no
heartbeat/animation beyond a spinner chip (IngestPanel.tsx:337-358).",
  "proposed_fix": "SPA half is Claude-doable now: persist the active job_id (?job=<id> +
localStorage) and on load resume polling to the existing ?video= handoff; add elapsed-
time/heartbeat copy. Engine half (granular 'segmenting chunk 3/7' from
backend/pipeline/segmenters/gemini.py via the job-progress hook) is a small cross-repo change.
Per the issue this is 'the most likely way an early tester loses a working job'.",
  "effort": "M",
  "tracked_in": "khelsutra#111"
}

Repo root candidates: C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer,
C:/Users/avidu/Projects/khelsutra-guru/khelsutra, C:/Users/avidu/Projects/khelsutra-guru/sports-
data-collector, C:/Users/avidu/Projects/khelsutra-guru/rally-annotator,
C:/Users/avidu/Projects/khelsutra-guru/sports-obsreport (pick from the finding's repo field;
non-git dirs: 'C:/Users/avidu/Projects/khelsutra-guru/Annotation Setup',
C:/Users/avidu/Projects/khelsutra-guru/khelsutra-screencast).
VERIFY against the CURRENT working tree (just pulled to origin master/main): (1) does the cited
code actually exist and behave as claimed at file:line? (2) was it already fixed by a recent
commit (git log --oneline -15 -- <file>)? (3) is the severity honest - construct the concrete
failure scenario or downgrade; (4) is the proposed fix sane and correctly scoped; (5) is it
owner-gated per: alpha/serving go-live, Studio P10/P11 corpus-promotion/training,
product/strategy/licensing, real cloud spend, counsel ToS/DPA?
HARD RULES: READ-ONLY (no file modifications, no installs, no test-suite runs). gh read-only
commands allowed. Your reasoning MUST cite file:line you actually read. CONFIRMED requires you
personally saw the defect/debt in current code; if you cannot reproduce the claim from the code,
say REFUTED; if a commit fixed it, ALREADY_FIXED.
```

2. user (2026-07-02T02:58:13.706Z)

```
f9eea14 web: resilient video-summary readers + show fps in the library (#140)
9223ba9 docs: refresh khelsutra docs for today's storage-track work (P2-P5) (#139)
fb2f561 web: persistent "FFmpeg not detected" banner (hard-blocker warning) (#138)
8667dbf P5: SPA storage-medium picker (store a video into your medium) (#137)
a04a21b docs(tracker): inline the D14 decisions into the P2-P5 rows (#136)
```

master...origin/master

3. user (2026-07-02T02:58:14.211Z)

```
1       // Drives the front-half ingest: submit a video to POST /api/process, then poll GET
/api/jobs/{id}
2       // until it finishes, and hand the resulting video_id to the existing rally-loading
back-half.
3       //
4       // Kept i18n-FREE (returns stable IngestErrorCodes, not prose) so it unit-tests without
a
5       // translation context ? IngestPanel maps codes to localized strings. The poll loop is a
RECURSIVE
6       // setTimeout (never setInterval): each tick waits for the prior getJob to resolve, so a
slow engine
7       // can't pile up overlapping requests. It backs off, gives up after a ceiling, and is
fully
8       // cancellable on unmount / user-cancel / a superseding submit.
9       import { useCallback, useEffect, useRef, useState } from "react";
10      import { getJob, processVideo } from "../api/client";
11      import { classifyError, type IngestError } from "../lib/errors";
12      import { log, newCorrelationId } from "../lib/log";
13      import type { JobStatus } from "../api/types";
14
15      const POLL_MIN_MS = 1000;
16      const POLL_MAX_MS = 5000;
17      const POLL_BACKOFF = 1.5;
18      const POLL_CEILING_MS = 10 * 60 * 1000; // a wedged 'running' job won't poll forever
19
20      export type IngestPhase =
21        | { status: "idle" }
22        | { status: "submitting" }
23        | { status: "polling"; progress: string | null }
24        | { status: "done"; videoId: string }
25        | { status: "error"; error: IngestError };
26
27      interface Run {
28        cancelled: boolean;
29        timer?: ReturnType<typeof setTimeout>;
30      }
31
32      export function useIngestJob(onLoaded?: (videoId: string) => void) {
33        const [phase, setPhase] = useState<IngestPhase>({ status: "idle" });
34        const runRef = useRef<Run | null>(null);
35        // Keep the latest onLoaded without making submit depend on it (a parent re-render
won't restart polls).
36        const onLoadedRef = useRef(onLoaded);
37        onLoadedRef.current = onLoaded;
38
39        const stop = useCallback(() => {
40          const run = runRef.current;
41          if (run) {
42            run.cancelled = true;
43            if (run.timer) clearTimeout(run.timer);
44          }
45          runRef.current = null;
46        }, []);
47
48        // Cancel any in-flight poll loop when the component unmounts (App.tsx:80-82 alive-
```

```
flag idiom).
49        useEffect(() => stop, [stop]);
50
51        const submit = useCallback(
52          (rawUri: string, locale?: string, compute?: string) => {
53            const uri = rawUri.trim();
54            if (!uri) {
55              setPhase({ status: "error", error: { code: "invalidUri" } });
56              return;
57            }
58
59            stop(); // supersede any prior run
60            const run: Run = { cancelled: false };
61            runRef.current = run;
62
63            const cid = newCorrelationId();
64            const startedAt = Date.now();
65            setPhase({ status: "submitting" });
66            log("info", "ingest.submit_start", { cid, uri });
67
68            const fail = (error: IngestError, event: string) => {
69              if (run.cancelled) return;
70              setPhase({ status: "error", error });
71              const level = error.code === "timeout" || error.code === "emptyResult" ? "warn"
: "error";
72              log(level, event, { cid, uri, code: error.code, detail: error.detail });
73            };
74
75            const poll = (jobId: string, delay: number) => {
76              run.timer = setTimeout(() => {
77                void (async () => {
78                  if (run.cancelled) return;
79                  let job: JobStatus;
80                  try {
81                    job = await getJob(jobId, cid);
82                  } catch (err) {
83                    fail(classifyError(err), "ingest.poll_error");
84                    return;
85                  }
86                  if (run.cancelled) return;
87
88                  if (job.status === "done") {
89                    const videoId = job.video_id ?? job.result?.video_id ?? null;
90                    // Two status layers: a `done` job with a `failed` pipeline verdict (or no
video_id) is
91                    // a success-shaped FAILURE ? never load it, never hang.
92                    if (job.result?.status === "failed" || !videoId) {
93                      fail({ code: "emptyResult" }, "ingest.empty_result");
94                      return;
95                    }
96                    setPhase({ status: "done", videoId });
97                    log("info", "ingest.done", { cid, uri, video_id: videoId });
98                    onLoadedRef.current?.(videoId);
99                    return;
100                 }
101                 if (job.status === "failed") {
102                   fail({ code: "engine", detail: job.error ?? "job failed" },
"ingest.job_failed");
103                   return;
104                 }
105                 // queued | running ? reflect coarse progress (free-text phase) and keep
polling.
106                 setPhase({ status: "polling", progress: job.progress ?? null });
107                 if (Date.now() - startedAt > POLL_CEILING_MS) {
108                   fail({ code: "timeout" }, "ingest.poll_timeout");
```

```
                  return;
                }
                poll(jobId, Math.min(delay * POLL_BACKOFF, POLL_MAX_MS));
              })();
            }, delay);
          };

          void (async () => {
            let jobId: string;
            try {
              const resp = await processVideo(uri, cid, locale, compute);
              jobId = resp.job_id;
            } catch (err) {
              fail(classifyError(err), "ingest.submit_error");
              return;
            }
            if (run.cancelled) return;
            log("info", "ingest.queued", { cid, uri, job_id: jobId });
            setPhase({ status: "polling", progress: null });
            poll(jobId, POLL_MIN_MS);
          })();
        },
        [stop],
      );

      const cancel = useCallback(() => {
        stop();
        setPhase({ status: "idle" });
        log("info", "ingest.cancel", {});
      }, [stop]);

      const reset = useCallback(() => {
        stop();
        setPhase({ status: "idle" });
      }, [stop]);

      const busy = phase.status === "submitting" || phase.status === "polling";
      return { phase, submit, cancel, reset, busy };
    }

```

4. user (2026-07-02T02:58:21.838Z)

```
f9eea14 web: resilient video-summary readers + show fps in the library (#140)
fb2f561 web: persistent "FFmpeg not detected" banner (hard-blocker warning) (#138)
cc06c32 feat(web): P8 annotate-in-UI ? AnnotatePanel over the vendored core (flagged) (#128)
9e9df72 feat(web): keep the video name visible after open (persistent current-video label)
(#127)
39c1349 feat(web): P6 ? "Your videos" library (reopen a stored video) (#124)
f67dfe7 feat(ingest): compute-picker ? choose local vs cloud GPU per run (item 3b) (#104)
7ae9cee feat(web): P2c first-run setup wizard (3 steps from /api/capabilities, 6 languages)
(#88)
c1848df feat(web): LW2 ? send the UI locale to /api/compile (watermark) + /api/process
(analytics) (#73)
46e0244 feat(web): A9 ? localized tap-only filter controls (#68)
501c014 feat(web): A5 ? friendly engine-error mapping across build + load (shared errors.*)
(#62)
95195a5 feat(web): A4 ? mobile touch targets (?44px) + visible tooltip text (#61)
4525c5d feat(web): B-P2 ? in-UI bucket ingest (point at a gs:// video ? process ? load) (#56)
289324e feat(web): extract all SPA strings to the en catalog + i18n parity CI ? tracker B2 (#45)
5b95b0d feat(web): i18n scaffold + language switcher (en-only) ? tracker B1 (#42)
155153d feat(web): lead with rally length; park shot-count as experimental (owner) (#33)
OPEN | Resilient processing UX: survive page refresh + continually-updating progress
```

5. user (2026-07-02T02:58:21.845Z)

```
      import { useEffect, useMemo, useState } from "react";
      import { useTranslation, Trans } from "react-i18next";
      import { useRallySelection } from "./hooks/useRallySelection";
      import { RallyGrid } from "./components/RallyGrid";
      import { QueryBar } from "./components/QueryBar";
      import { TapFilters } from "./components/TapFilters";
      import { SelectionFooter } from "./components/SelectionFooter";
      import { LanguageSwitcher } from "./components/LanguageSwitcher";
      import { CompileResult, type BuildState } from "./components/CompileResult";
      import { IngestPanel } from "./components/IngestPanel";
      import { FfmpegBanner } from "./components/FfmpegBanner";
      import { VideoLibrary } from "./components/VideoLibrary";
      import { AnnotatePanel } from "./components/AnnotatePanel";
      import { SetupWizard, isWizardDone } from "./components/SetupWizard";
      import { runQuery, type ParsedQuery } from "./lib/query";
      import { floorRisk } from "./lib/floor";
      import { reelFilename } from "./lib/format";
      import { classifyError } from "./lib/errors";
      import { errorText } from "./lib/errorText";
      import { log, newCorrelationId } from "./lib/log";
      import { ApiError, compileReel, getConfig, highlightUrl, listRallies, listVideos,
minShotCount } from "./api/client";
      import { videoId as videoIdOf, videoName } from "./lib/video";
      import type { EngineConfig, RallySegment } from "./api/types";

      /** One compact, structured description of how a query was understood (for the trace
log). */
      function parseTrace(q: ParsedQuery) {
        return {
          raw: q.raw,
          recognized: q.recognized,
          filters: q.filters.map((f) => `${f.field}${f.op}${f.value}`),
          sort: q.sort ? `${q.sort.key}:${q.sort.dir}` : null,
          limit: q.limit ?? null,
          unavailable: q.unavailable.map((u) => `${u.kind}:${u.token}`),
        };
      }

      // Demo data shown when no ?video=<id> is supplied. Shape matches the verified
play_segments
      // contract; only honest fields are surfaced. A real session loads from POST /api/query.
      const DEMO_RALLIES: RallySegment[] = [
        { id: 1, start_time: 12, end_time: 20, duration: 8, server: null, receiver: null,
metadata: { shot_count: 9, shot_count_confidence: "High" } },
        { id: 2, start_time: 41, end_time: 47, duration: 6, server: null, receiver: null,
metadata: { shot_count: 5 } },
        { id: 3, start_time: 63, end_time: 84, duration: 21, server: null, receiver: null,
metadata: { shot_count: 19, shot_count_confidence: "Medium" } },
        { id: 4, start_time: 95, end_time: 101, duration: 6, server: null, receiver: null,
metadata: {} },
        { id: 5, start_time: 130, end_time: 142, duration: 12, server: null, receiver: null,
metadata: { shot_count: 11 } },
        { id: 6, start_time: 158, end_time: 187, duration: 29, server: null, receiver: null,
metadata: { shot_count: 24, shot_count_confidence: "High" } },
      ];

      type LoadState =
        | { status: "demo" }
        | { status: "loading" }
        | { status: "loaded"; count: number }
        | { status: "error"; error: string };

      /** Annotate-in-UI (P8) is behind a runtime feature flag ? default OFF (D13: ships gated
until consented
       *  / no-minors footage is in hand). Turn it on per-browser with `?annotate=1` or
```

```
56       *  `localStorage['khelsutra.annotate'] = '1'`. No build step; the owner enables it when
ready. */
57      function annotateFlag(): boolean {
58        try {
59          if (new URLSearchParams(window.location.search).get("annotate") === "1") return
true;
60          return localStorage.getItem("khelsutra.annotate") === "1";
61        } catch {
62          return false;
63        }
64      }
65
66      export default function App() {
67        const { t, i18n } = useTranslation();
68        // Job-tethered (D10): a finished job's video_id arrives via ?video=<id> ? seeded from
the URL,
69        // but now also SETTABLE so the in-UI ingest panel (B-P2) can hand off a freshly-
processed video
70        // without a reload (it also rewrites ?video= so a refresh is stable).
71        const [videoId, setVideoId] = useState<string | null>(
72          () => new URLSearchParams(window.location.search).get("video"),
73        );
74
75        const [rallies, setRallies] = useState<RallySegment[]>(videoId ? [] : DEMO_RALLIES);
76        const [order, setOrder] = useState<RallySegment[]>(videoId ? [] : DEMO_RALLIES);
77        const [loadState, setLoadState] = useState<LoadState>(videoId ? { status: "loading" }
: { status: "demo" });
78        const [reelName, setReelName] = useState("highlights");
79        const [build, setBuild] = useState<BuildState>({ status: "idle" });
80        // P2c: the first-run wizard shows once per browser, before the ingest panel, when
nothing's loaded.
81        const [wizardDone, setWizardDone] = useState(isWizardDone);
82        // The human name of the loaded video ? shown PERSISTENTLY so the operator always
knows which match
83        // they're editing. Once a video loads, the library + ingest panel (which showed the
name) unmount,
84        // so without this the name would vanish. Set from the library on click; else resolved
below.
85        const [videoLabel, setVideoLabel] = useState<string | null>(null);
86        // P8 (flagged): switch the loaded-video view between picking rallies and annotating
them.
87        const [annotateEnabled] = useState(annotateFlag);
88        const [view, setView] = useState<"pick" | "annotate">("pick");
89
90        // Display order is separate from selection: a query can re-sort the cards while the
shared
91        // selection set (sel) stays the single source of truth for what lands in the reel.
92        const sel = useRallySelection(rallies, "all");
93
94        // Load real rallies for a job's video (POST /api/query). Default ALL-selected (D8).
95        useEffect(() => {
96          if (!videoId) return;
97          const cid = newCorrelationId();
98          log("info", "rallies.load_start", { cid, video_id: videoId });
99          // Show "loading" even when videoId was set dynamically by the ingest panel (initial
state was
100         // "demo" in that path) ? the banner must reflect the load, not stale demo copy.
101         setLoadState({ status: "loading" });
102         let alive = true;
103         listRallies(videoId, cid)
104           .then((rs) => {
105             if (!alive) return;
106             setRallies(rs);
107             setOrder(rs);
108             sel.apply(rs.map((r) => r.id));
```

```
109             setLoadState({ status: "loaded", count: rs.length });
110             log("info", "rallies.loaded", { cid, video_id: videoId, count: rs.length });
111           })
112           .catch((err) => {
113             if (!alive) return;
114             // Friendly, localized ? same shared mapping as build/ingest (A5): a load
failure on a slow or
115             // down engine reads "is it running?", not a raw "ApiError: Network error
calling POST ?".
116             const e = classifyError(err);
117             setLoadState({ status: "error", error: errorText(t, e) });
118             log("error", "rallies.load_failed", { cid, video_id: videoId, code: e.code,
detail: e.detail, error: String(err) });
119           });
120         return () => {
121           alive = false;
122         };
123         // videoId is stable for the page; sel.apply is a stable callback.
124         // eslint-disable-next-line react-hooks/exhaustive-deps
125       }, [videoId]);
126
127       // Read the engine's stitching.min_shot_count so the footer can WARN before a selected
rally is
128       // silently dropped at compile (P4 / D7). Offline (no engine) is fine: no config ? no
warning.
129       const [cfg, setCfg] = useState<EngineConfig | null>(null);
130       useEffect(() => {
131         let alive = true;
132         getConfig()
133           .then((c) => alive && setCfg(c))
134           .catch((err) => log("warn", "config.unavailable", { error: String(err) }));
135         return () => {
136           alive = false;
137         };
138       }, []);
139       const warning = floorRisk(rallies, sel.selected, cfg ? minShotCount(cfg) : undefined);
140
141       // Stable chronological ordinal (by start_time) so re-sorting never renumbers a rally.
142       const ordinalOf = useMemo(() => {
143         const m = new Map<number, number>();
144         [...rallies].sort((a, b) => a.start_time - b.start_time).forEach((r, i) =>
m.set(r.id, i + 1));
145         return (id: number) => m.get(id) ?? 0;
146       }, [rallies]);
147
148       // Load a video into the picker ? from a fresh ingest (IngestPanel) OR reopening a
stored one
149       // (VideoLibrary, which passes the match name). Rewrites ?video= so a refresh is
stable (D10).
150       const loadVideo = (id: string, label?: string) => {
151         setVideoId(id);
152         setVideoLabel(label ?? null);
153         window.history.replaceState(null, "", `?video=${encodeURIComponent(id)}`);
154       };
155
156       // Resolve the loaded video's name when we don't already have it (a deep-link ?video=
on reload, or
157       // an ingest hand-off) so the current-video label is always populated ? best-effort,
from the same
158       // GET /api/videos the library uses. A miss (not yet listed) keeps the friendly
fallback.
159       useEffect(() => {
160         if (!videoId || videoLabel) return;
161         let alive = true;
162         listVideos()
```

```
          .then((vs) => {
            const v = vs.find((x) => videoIdOf(x) === videoId);
            const name = v ? videoName(v) : "";
            if (alive && name) setVideoLabel(name);
          })
          .catch((err) => log("warn", "video.name_resolve_failed", { video_id: videoId,
error: String(err) }));
        return () => {
          alive = false;
        };
      }, [videoId, videoLabel]);

      // Build ? POST /api/compile ? preview/download the stitched reel. One cid traces the
whole call.
      const onBuild = async () => {
        if (build.status === "building") return;
        const cid = newCorrelationId();
        const filename = reelFilename(reelName);
        const ids = [...sel.selected];
        if (!videoId) {
          log("info", "build.no_video", { cid, selected: ids.length });
          setBuild({
            status: "error",
            error: t("build.noVideo"),
          });
          return;
        }
        setBuild({ status: "building" });
        log("info", "build.start", {
          cid,
          video_id: videoId,
          filename,
          selected: ids.length,
          total_sec: Math.round(sel.totalSec),
          below_floor: warning?.atRisk.length ?? 0,
        });
        try {
          // Pass the active UI locale so the engine localizes the reel's branding watermark
(LW1).
          const res = await compileReel(videoId, ids, filename, cid, i18n.resolvedLanguage
?? i18n.language);
          const url = highlightUrl(videoId, filename);
          setBuild({ status: "done", url, filename });
          log("info", "build.done", { cid, video_id: videoId, filename, filepath:
res.filepath, url });
        } catch (err) {
          // Same friendly mapping the ingest flow uses: a bare 5xx / network throw reads
"is the engine
          // running?", a real engine 4xx surfaces its own detail ? never a raw "HTTP 500"
(A5).
          const status = err instanceof ApiError ? err.status : undefined;
          const e = classifyError(err);
          setBuild({ status: "error", error: errorText(t, e) });
          log("error", "build.failed", { cid, video_id: videoId, filename, status, code:
e.code, detail: e.detail, error: String(err) });
        }
      };

      // Every query application is ONE correlatable journey: a `query.parse` line records
exactly how
      // the text was understood, then a single outcome line (apply / skip / empty) shares
the same cid.
      // Unsupported concepts and zero-result queries are logged loudly ? never silently
dropped.
      const onApplyQuery = (q: ParsedQuery, source: "typed" | "example") => {
```

```
        const cid = newCorrelationId();
        log("info", "query.parse", { cid, source, ...parseTrace(q) });

        // Demand signal + honest "we ignored this": surface every shot-type/player/score
the user asked
        // for but we can't yet honour (PER_RALLY_SELECTION.md §3 roadmap fields).
        if (q.unavailable.length > 0) {
          log("info", "query.unavailable_request", { cid, source, raw: q.raw, unavailable:
q.unavailable });
        }

        if (!q.recognized) {
          log("info", "query.skip", { cid, source, raw: q.raw, reason: "no actionable
clause" });
          return; // empty / unparseable ? leave the user's curation untouched
        }

        try {
          const { ordered, selectedIds } = runQuery(rallies, q);
          setOrder(ordered);
          sel.apply(selectedIds);
          log("info", "query.apply", {
            cid,
            source,
            filters: q.filters.length,
            sort: q.sort ? `${q.sort.key}:${q.sort.dir}` : null,
            limit: q.limit ?? null,
            total: rallies.length,
            selected: selectedIds.length,
            selected_ids: selectedIds,
            dropped_unavailable: q.unavailable.map((u) => `${u.kind}:${u.token}`),
          });
          if (selectedIds.length === 0) {
            // Loud: a recognized query that matches nothing makes the grid look empty ? say
why.
            log("warn", "query.empty_result", { cid, source, ...parseTrace(q) });
          }
        } catch (err) {
          // Defensive: the pure parser/selector shouldn't throw, but never fail silently if
it does.
          log("error", "query.apply_failed", { cid, source, raw: q.raw, error: String(err)
});
        }
      };

      // Manual curation edits are traced too, so a journey (query ? hand-tune) reads as one
story.
      const onToggle = (id: number) => {
        log("debug", "rally.toggle", { id, action: sel.isSelected(id) ? "remove" : "add" });
        sel.toggle(id);
      };
      const onBulk = (op: "select_all" | "clear" | "invert") => {
        const resultCount = op === "select_all" ? rallies.length : op === "clear" ? 0 :
rallies.length - sel.count;
        log("info", "selection.bulk", { cid: newCorrelationId(), op, result_count:
resultCount });
        if (op === "select_all") sel.selectAll();
        else if (op === "clear") sel.clearAll();
        else sel.invert();
      };

      return (
        <main className="min-h-screen bg-paper text-ink pb-24">
          <header className="bg-ink text-white">
            <div className="mx-auto flex max-w-5xl items-center justify-between gap-3 px-6
```

```
py-4">
273               <span className="text-xl font-extrabold tracking-tight">
274                 Khel<span className="text-green">?</span>Sutra <span className="font-
semibold text-lime">Studio</span>
275               </span>
276               <LanguageSwitcher />
277             </div>
278           </header>
279
280           <section className="mx-auto max-w-5xl px-6 py-10">
281             {/* Hard-blocker warning: FFmpeg absent ? indexing + reel compile can't run.
Persistent (unlike
282                 the once-per-browser wizard), shown only when the engine positively reports
it missing. */}
283             <FfmpegBanner />
284             <span className="inline-flex rounded-full bg-green/15 px-3 py-1 text-xs font-
semibold text-green">
285               {t("app.badge")}
286             </span>
287             <h1 className="mt-4 text-3xl font-extrabold tracking-
tight">{t("app.title")}</h1>
288             <p className="mt-2 max-w-2xl text-ink/70">
289               <Trans i18nKey="app.intro" components={{ b: <strong />, i: <em /> }} />
290             </p>
291
292             {/* Persistent "current video" label ? always visible once a video is loaded, so
the operator
293                 never loses track of which match they're editing (the library/ingest panel
that showed the
294                 name has unmounted by now). Name from the library, else resolved from GET
/api/videos. */}
295             {videoId && (
296               <p
297                 className="mt-3 inline-flex items-center gap-2 rounded-full bg-ink/5 px-3
py-1 text-sm text-ink/70"
298                 data-testid="current-video"
299               >
300                 <span aria-hidden="true">?</span>
301                 <Trans
302                   i18nKey="app.currentVideo"
303                   values={{ name: videoLabel ?? t("app.currentVideoUnknown") }}
304                   components={{ b: <strong className="text-ink" /> }}
305                 />
306               </p>
307             )}
308
309             {/* P8 (flagged): switch the loaded video between picking rallies and annotating
them. */}
310             {videoId && annotateEnabled && (
311               <div className="mt-4 inline-flex rounded-full border border-black/10 p-1 text-
sm" data-testid="view-toggle">
312                 <button
313                   type="button"
314                   onClick={() => setView("pick")}
315                   aria-pressed={view === "pick"}
316                   className={"min-h-11 rounded-full px-4 " + (view === "pick" ? "bg-ink
text-white" : "text-ink/70")}
317                 >
318                   {t("annotate.tabPick")}
319                 </button>
320                 <button
321                   type="button"
322                   onClick={() => setView("annotate")}
323                   aria-pressed={view === "annotate"}
324                   className={"min-h-11 rounded-full px-4 " + (view === "annotate" ? "bg-ink
```

```
text-white" : "text-ink/70")}
                >
                  {t("annotate.tab")}
                </button>
              </div>
            )}

            {/* P2c first-run wizard: before anything is loaded, walk a new user through AI-
key / compute /
                videos (from /api/capabilities). Shows once per browser; then the ingest
panel takes over. */}
            {!videoId && !wizardDone && <SetupWizard onClose={() => setWizardDone(true)} />}

            {/* Front-half (P6 + B-P2): when no video is loaded, offer BOTH ? reopen a
stored video from
                "Your videos" (VideoLibrary ? GET /api/videos), or add a new one
(IngestPanel ? process ?
                load). Once a video_id exists, the existing back-half (pick ? build) takes
over. */}
            {!videoId && wizardDone && (
              <>
                <VideoLibrary onSelect={loadVideo} />
                <IngestPanel onLoaded={loadVideo} />
              </>
            )}

            {/* P8 annotate view (flagged): the labeler over the loaded video. */}
            {view === "annotate" && videoId && <AnnotatePanel key={videoId}
videoId={videoId} />}

            {/* Pick-rallies view (the default + the whole app when annotate is off). */}
            {view === "pick" && (
              <>
                <ModeBanner load={loadState} videoId={videoId} />

                <div className="mt-6">
                  <QueryBar onApply={onApplyQuery} />
                  {/* A9: localized tap controls for users who can't type the English query
syntax. */}
                  <TapFilters onApply={onApplyQuery} />
                </div>

                <div className="mt-6 flex flex-wrap gap-2 text-sm">
                  <button type="button" onClick={() => onBulk("select_all")}
className="inline-flex min-h-11 items-center rounded-full border border-black/15 px-4
hover:border-green">{t("bulk.selectAll")}</button>
                  <button type="button" onClick={() => onBulk("clear")} className="inline-
flex min-h-11 items-center rounded-full border border-black/15 px-4 hover:border-
green">{t("bulk.clear")}</button>
                  <button type="button" onClick={() => onBulk("invert")} className="inline-
flex min-h-11 items-center rounded-full border border-black/15 px-4 hover:border-
green">{t("bulk.invert")}</button>
                </div>

                <div className="mt-5">
                  <RallyGrid rallies={order} isSelected={sel.isSelected} onToggle={onToggle}
ordinalOf={ordinalOf} />
                </div>

                <CompileResult build={build} onDismiss={() => setBuild({ status: "idle" })}
/>
              </>
            )}
          </section>

```

```
374           {/* The reel-building footer belongs to the pick view only (annotation has its own
save). */}
375           {view === "pick" && (
376             <SelectionFooter
377               count={sel.count}
378               totalSec={sel.totalSec}
379               reelName={reelName}
380               onReelNameChange={setReelName}
381               onBuild={onBuild}
382               building={build.status === "building"}
383               warning={warning}
384             />
385           )}
386         </main>
387       );
388     }
389
390     function ModeBanner({ load, videoId }: { load: LoadState; videoId: string | null }) {
391       if (load.status === "demo") {
392         return (
393           <p className="mt-3 text-sm text-ink/50" data-testid="mode-banner">
394             <Trans i18nKey="mode.demo" components={{ code: <code /> }} />
395           </p>
396         );
397       }
398       if (load.status === "loading") {
399         return (
400           <p className="mt-3 text-sm text-ink/60" data-testid="mode-banner">
401             <Trans i18nKey="mode.loading" values={{ videoId }} components={{ code: <code />
}} />
402           </p>
403         );
404       }
405       if (load.status === "error") {
406         return (
407           <p className="mt-3 rounded-lg border border-red-200 bg-red-50 px-3 py-2 text-sm
text-red-800" data-testid="mode-banner">
408             <Trans i18nKey="mode.error" values={{ videoId, error: load.error }}
components={{ code: <code /> }} />
409           </p>
410         );
411       }
412       return (
413         <p className="mt-3 text-sm text-ink/50" data-testid="mode-banner">
414           <Trans i18nKey="mode.loaded" values={{ count: load.count }} components={{ b:
<strong className="text-ink/70" /> }} />
415         </p>
416       );
417     }
418
```

6. user (2026-07-02T02:58:34.806Z)

```
khelsutra\web\src\App.tsx:56: *  `localStorage['khelsutra.annotate'] = '1'`. No build step; the
owner enables it when ready. */
khelsutra\web\src\App.tsx:59:    if (new URLSearchParams(window.location.search).get("annotate")
=== "1") return true;
khelsutra\web\src\App.tsx:60:    return localStorage.getItem("khelsutra.annotate") === "1";
khelsutra\web\src\App.tsx:72:    () => new URLSearchParams(window.location.search).get("video"),
khelsutra\web\src\App.tsx:153:    window.history.replaceState(null, "",
`?video=${encodeURIComponent(id)}`);
khelsutra\web\src\test\setup.ts:2:// expose a Web Storage `localStorage`, which the i18n
language detector and the persistence test
khelsutra\web\src\test\setup.ts:4:if (typeof window !== "undefined" && typeof
globalThis.localStorage === "undefined") {
```

```
khelsutra\web\src\test\setup.ts:26:  Object.defineProperty(window, "localStorage", { value: mem,
configurable: true, writable: true });
khelsutra\web\src\test\setup.ts:27:  Object.defineProperty(globalThis, "localStorage", { value:
mem, configurable: true, writable: true });
khelsutra\web\src\i18n\index.ts:60:      order: ["localStorage", "querystring", "navigator"],
khelsutra\web\src\i18n\index.ts:63:      caches: ["localStorage"],
khelsutra\web\src\i18n\i18n.test.tsx:15:  window.localStorage.clear();
khelsutra\web\src\i18n\i18n.test.tsx:47:
expect(window.localStorage.getItem(LANG_STORAGE_KEY)).toBe("kn");
khelsutra\web\src\i18n\i18n.test.tsx:92:    window.localStorage.clear(); // no persisted
preference yet
khelsutra\web\src\i18n\i18n.test.tsx:98:    window.localStorage.clear();
khelsutra\web\src\i18n\i18n.test.tsx:104:    window.localStorage.setItem(LANG_STORAGE_KEY,
"kn");
khelsutra\web\src\i18n\i18n.test.tsx:111:
expect(order.indexOf("localStorage")).toBeGreaterThanOrEqual(0);
khelsutra\web\src\i18n\i18n.test.tsx:112:
expect(order.indexOf("localStorage")).toBeLessThan(order.indexOf("navigator"));
khelsutra\web\src\i18n\i18n.test.tsx:117:    window.localStorage.clear();
khelsutra\web\src\i18n\i18n.test.tsx:165:      window.localStorage.clear();
khelsutra\web\src\components\build-error.test.tsx:39:  window.localStorage.clear();
khelsutra\web\src\components\a4.touch.test.tsx:21:  window.localStorage.clear();
khelsutra\web\src\components\a4.touch.test.tsx:24:  window.localStorage.setItem(WIZARD_DONE_KEY,
"true");
khelsutra\web\src\components\a8.status.test.tsx:13:  window.localStorage.clear();
khelsutra\web\src\components\QueryBar.test.tsx:13:  window.localStorage.clear();
khelsutra\web\src\components\LanguageSwitcher.tsx:17: * detector caches to localStorage
(LANG_STORAGE_KEY) ? so the choice is the top-priority signal on
khelsutra\web\src\components\SelectionFooter.test.tsx:25:    window.localStorage.clear();
khelsutra\web\src\components\SelectionFooter.test.tsx:57:  beforeEach(() =>
window.localStorage.clear());
khelsutra\web\src\components\SelectionFooter.test.tsx:89:    window.localStorage.clear();
khelsutra\web\src\components\IngestPanel.test.tsx:62:  window.localStorage.clear();
khelsutra\web\src\components\SetupWizard.test.tsx:36:  window.localStorage.clear();
khelsutra\web\src\components\SetupWizard.test.tsx:75:
expect(window.localStorage.getItem(WIZARD_DONE_KEY)).toBe("true");
khelsutra\web\src\components\SetupWizard.test.tsx:110:  it("isWizardDone reads the localStorage
flag", () => {
khelsutra\web\src\components\SetupWizard.test.tsx:112:
window.localStorage.setItem(WIZARD_DONE_KEY, "true");
khelsutra\web\src\components\SetupWizard.tsx:11:/** localStorage flag ? distinct from the
language key ? so the wizard shows ONCE per browser. */
khelsutra\web\src\components\SetupWizard.tsx:16:    return localStorage.getItem(WIZARD_DONE_KEY)
=== "true";
khelsutra\web\src\components\SetupWizard.tsx:109:      localStorage.setItem(WIZARD_DONE_KEY,
"true");
khelsutra\web\src\components\upload.test.tsx:51:  window.localStorage.clear();
```

7. user (2026-07-02T02:58:35.331Z)

```
320                   </span>
321                 </div>
322                 <p className="mt-2 text-xs text-ink/50">{t("ingest.reassure")}</p>
323                 <button
324                   type="button"
325                   onClick={cancelAll}
326                   className="mt-2 inline-flex min-h-11 items-center rounded-xl border
border-black/15 px-4 text-sm hover:border-green"
327                 >
328                   {t("ingest.cancel")}
329                 </button>
330               </div>
331             )}
332             {storing && (
333               <div data-testid="ingest-storing">
```

```
334                 <StatusChip tone="info" icon={<Spinner />}
label={t("ingest.medium.storing")} />
335               </div>
336             )}
337             {ingestBusy && (
338               <div>
339                 <StatusChip
340                   tone="info"
341                   icon={<Spinner />}
342                   label={
343                     t("ingest.working") +
344                     (phase.status === "polling" && phase.progress
345                       ? ` · ${t("ingest.progress", { progress: phase.progress })}`
346                       : "")
347                   }
348                 />
349                 {/* Reassurance for slow uplinks: the job runs on the server, so it's safe
to wait. */}
350                 <p className="mt-2 text-xs text-ink/50">{t("ingest.reassure")}</p>
351                 <button
352                   type="button"
353                   onClick={cancelAll}
354                   className="mt-2 inline-flex min-h-11 items-center rounded-xl border
border-black/15 px-4 text-sm hover:border-green"
355                 >
356                   {t("ingest.cancel")}
357                 </button>
358               </div>
359             )}
360             {errorMsg && (
361               <div className="rounded-lg border border-red-200 bg-red-50 px-3 py-2 text-
red-800" data-testid="ingest-error">
362                 <StatusChip tone="error" icon="?" label={t("ingest.statusFailed")} />
363                 <p className="mt-2">{errorMsg}</p>
364                 <button
365                   type="button"
366                   onClick={resetAll}
367                   className="mt-2 inline-flex min-h-11 items-center rounded-lg border
border-red-300 px-3 text-sm hover:bg-red-100"
368                 >
369                   {t("ingest.retry")}
370                 </button>
371               </div>
372             )}
373           </div>
374         </section>
```

8. user (2026-07-02T02:59:05.936Z)

```
__init__.py
__pycache__
_keyhint.py
base.py
fusion_hybrid.py
gemini.py
motion.py
tracknet_hybrid.py
trajectory_hybrid.py
wasb_hybrid.py
yolo_hybrid.py
---
**Context (from the first live alpha run on the droplet, 2026-06-22).** A real ~8.3-min upload
took ~5+ min of Gemini segmentation (7 chunks of 90s). During that window the SPA shows a
coarse, near-static "processing" phase, and a **page refresh drops the user back to the ingest
form** even though the job keeps running server-side. Users naturally refresh when nothing
```

visibly moves ? and then think they've lost their work.

Already fine (do NOT re-solve)

The **backend is refresh-safe**: jobs run server-side and are persisted in the SQLite `jobs` table (the async job worker); a client refresh does **not** kill or affect the running job.

The gaps

1. **SPA loses the in-flight `job_id` on refresh.** `web/src/hooks/useIngestJob.ts` holds the active `job_id` only in the poll-loop's runtime state ? there's no `localStorage`/URL persistence. On refresh the client can't reconnect to the still-running job ? the user lands back on the ingest form. (On *done* it sets `?video=<id>`, but mid-job there's nothing to resume from.)
2. **Progress looks static for minutes.** The SPA reflects `job.progress` (a coarse free-text phase), but the Gemini segmenter *logs* per-chunk progress ("Extracting Chunk 3/7") and never pushes it to the job-progress hook ? so the phase text doesn't move during the multi-minute segmentation.

Requirements / acceptance

- [ ] **Survive refresh** ? persist the active `job_id` (URL `?job=<id>` and/or `localStorage`); on load, if a job is in-flight, **resume polling it** and reconnect to live progress, then hand off to the existing `?video=` flow on done. A refresh during processing must seamlessly re-attach to the running job, never lose it.
- [ ] **Continually-updating, reassuring progress** ? a heartbeat / elapsed-time indicator + an animated state so the user always sees movement even when the engine phase is unchanged. Copy along the lines of *"This can take a few minutes ? it's safe to leave or refresh this tab; you won't lose your place."*
- [ ] **(Engine, linked ? repo `Khelsutra/badminton-highlight-indexer`)** Report granular progress from the Gemini segmenter via the job-progress hook (e.g. `segmenting chunk 3/7`) so the SPA has real movement to show. Small change in `backend/pipeline/segmenters/gemini.py`.

Nice-to-haves

- Show partial results as chunks complete.
- A Cancel button.
- Surface the active/recent job in history so a returning user can find it.

Surfaced live while watching the first real alpha run ? a long Gemini job + a refresh-fragile client is the most likely way an early tester gets confused or "loses" a working job.

9. user (2026-07-02T02:59:43.281Z)

Structured output provided successfully